



UFSC - Universidade Federal de Santa Catarina
Centro Tecnológico de Joinville
EMB5615 - Robótica e Sistemas Mecatrônicos

Paletização simples com integração à CLP

Carlos Henrique Beckert
Fernando Mendes Peixer
Luiz Guilherme Albino de Souza
Priscila Tanno Nogueira de Carvalho

1 Introdução

Este Trabalho trata de desenvolver a simulação e também executar na prática paletização simples no UR10 real com integração à CLP do laboratório. Para isso utilizou-se o URSIM como mestre enviando os outputs e esperando inputs do CODESYS. Além disso, tem a simulação que passa a movimentação em tempo real dapalletizando para o ROBO DK por meio de um código no URSIM.

2 Paletização

Paletização é organizar um conjunto de caixas sobre um palete, seu desafio é executar essa ação de maneira eficiente. Maximizando o uso do espaço e respeitando restrições físicas (limite de peso, estabilidade da carga, fragilidade dos itens e orientações permitidas de empilhamento). Também inclui a eficácia da comunicação entre a CLP e o robô com os sinais de comando, estado e segurança.

3 Trilha 3, Integração entre robô e CLP

Escolheu-se a Trilha 3 – Paletização simples com integração à CLP do laboratório, que visa integrar o controle de um robô industrial (UR10) com sistemas de automação de bancada (CLP), permitindo o estudo prático de protocolos de comunicação e sincronismo em células robotizadas.

4 Arquitetura da solução

Utilizou-se CLP virtual do CODESYS para comunicação pelo protocolo Mod-Bus TCP via rede Ethernet. Em que CODESYS é o server, o URSIM é o cliente. Além disso, o software ROBO DK é usado para simulação da paletização.

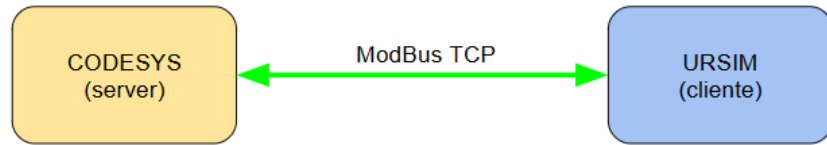


Figura 1: Diagrama de comunicação/ligação

5 Desenvolvimento

5.1 Referenciais, pontos e parâmetros

Os endereços utilizados entre URSim e CODESYS estão na Tabela 1.

Tabela 1: Mapeamento de endereços e fenômeno de inversão de bytes (Byte Swap) entre URSim e CODESYS.

Endereço Modbus (URSim)	Posição na Word (16 bits)	Bit Mapeado (CODESYS)
4	Byte Alto (High Byte)	12
5	Byte Alto (High Byte)	13
6	Byte Alto (High Byte)	14
7	Byte Alto (High Byte)	15
8	Byte Baixo (Low Byte)	0
9	Byte Baixo (Low Byte)	1
10	Byte Baixo (Low Byte)	2
11	Byte Baixo (Low Byte)	3
12	Byte Baixo (Low Byte)	4
13	Byte Baixo (Low Byte)	5

5.2 Lógica de paletização

A lógica de paletização, Figura 2, inicia-se o programa e a função *FreeDrive-Mode()* é executada e após isso entra em modo de espera que chegue a caixa na esteira. Então começa a rotina de paletização, quando essa encerrada se não forem duas camadas encerra. Se tiver mais uma camada volta para o modo de espera, executa a rotina de paletização e encerra. Quem defina a quantidade de camadas é o operador.

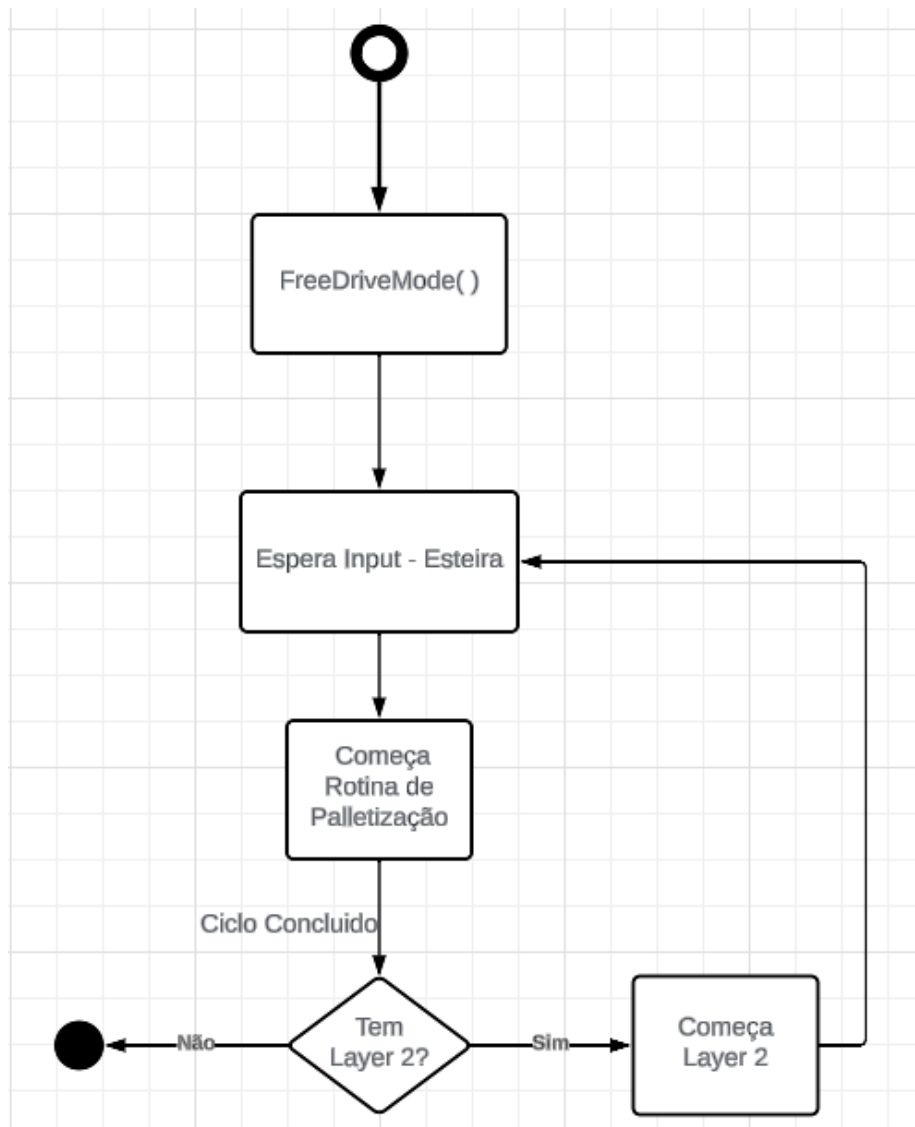


Figura 2: Máquina de estados

5.3 Integração específica da trilha escolhida

A validação em ambiente virtual e de simulação foi consolidada a partir da integração coesa entre três ferramentas de software: o CODESYS (Control Win V3 x64), atuando como o controlador lógico programável (CLP) virtual; o URSim, operando como o ambiente emulador nativo da Universal Robots para a execução do script de controle do braço robótico; e o RoboDK, utilizado para

o planejamento tridimensional e geração inicial das trajetórias cinemáticas do manipulador UR10.

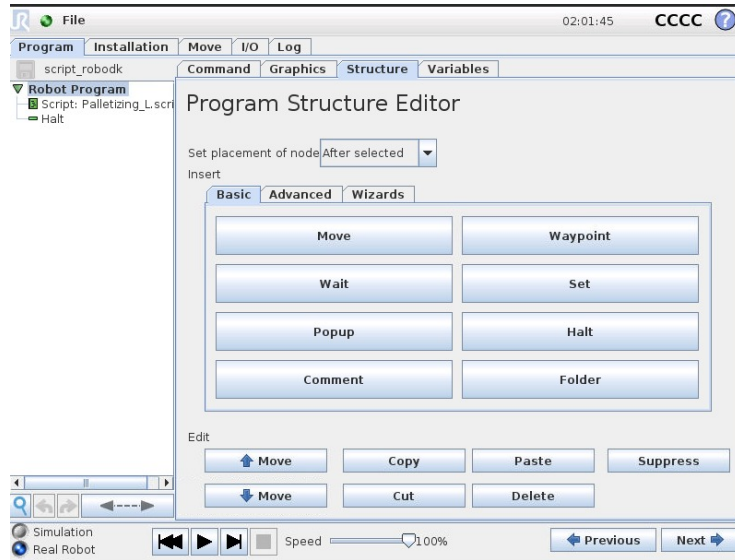


Figura 3: Árvore do projeto no URSIM com Script de paletização gerado pelo RoboDK

A arquitetura de comunicação foi estruturada sob o protocolo Modbus TCP, estabelecendo uma topologia onde o CODESYS foi configurado como dispositivo Slave (Servidor) e o URSim operou no papel de Client (Cliente).

Para viabilizar o tráfego e teste dos endereçamentos lógicos, foi adicionado um módulo escravo de rede Ethernet no CODESYS, parametrizado com uma janela expandida de 100 registros de retenção (Holding Registers, mapeados em %IW) e 100 registros de entrada (Input Registers, mapeados em %QW). Essa abordagem baseada em palavras de 16 bits (Words) permitiu a otimização do barramento de dados: em vez de trafegar bits isolados por requisições distintas, as flags operacionais e os comandos de intertravamento da célula (como botões de acionamento e indicadores de status) foram mapeados como bits específicos (Bit Picking) contidos dentro de arrays de palavras (por exemplo, UREntrada1 e UREntrada2). Dessa forma, a alteração de nível lógico em uma única variável booleana na lógica em linguagem Ladder do CODESYS resulta na transmissão instantânea do valor decimal equivalente da palavra inteira pela rede.

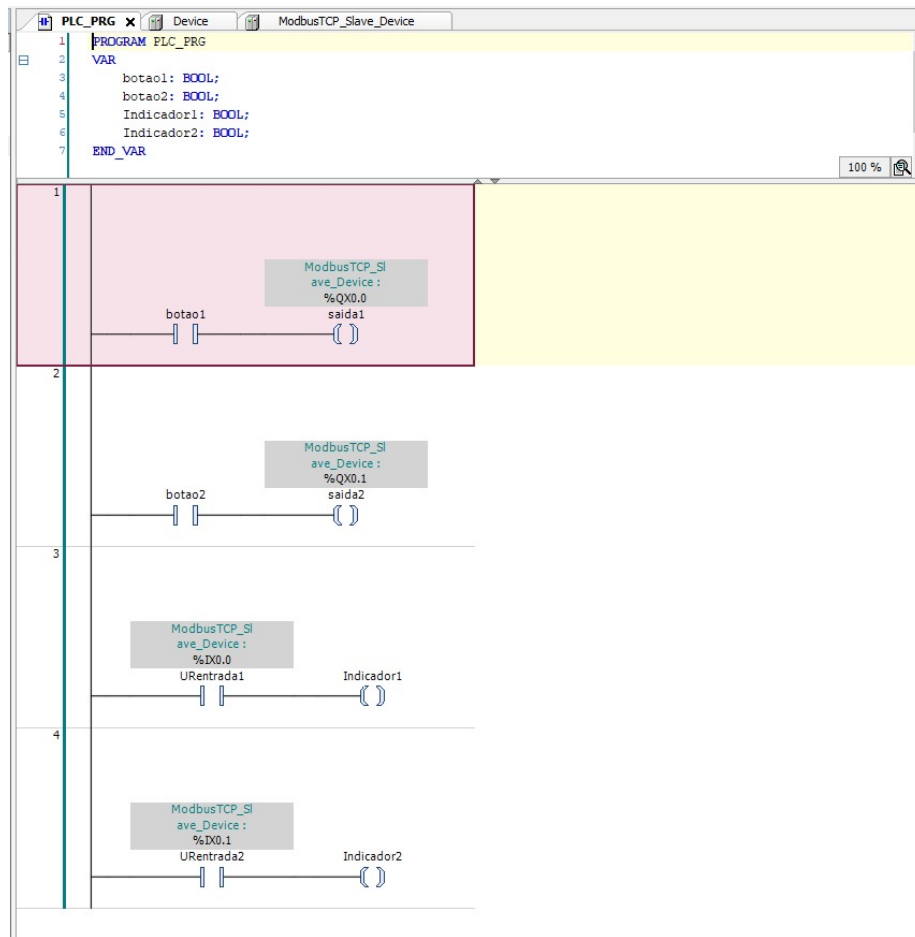


Figura 4: Código CODESYS

Do lado do cliente, a interface de controle do URSim foi configurada apontando diretamente para o endereço IPv4 local do host (identificado via adaptador de rede virtual como 192.168.240.1 na porta padrão 502). Na aba de configuração de I/O do Modbus, foram criadas as variáveis de vinculação para os campos de Register Input e Register Output, realizando a leitura síncrona das Words enviadas pelo CLP e salvando-as em variáveis internas (como coil e MODBUS). Essa estrutura garantiu que o programa principal em URScript — originalmente projetado no RoboDK e importado diretamente para a árvore de blocos do programa no Polyscope do URSim — pudesse tomar decisões de movimento em tempo real e chavear as saídas de status de forma síncrona com a varredura da lógica Ladder.

Figura 5: Enter Caption

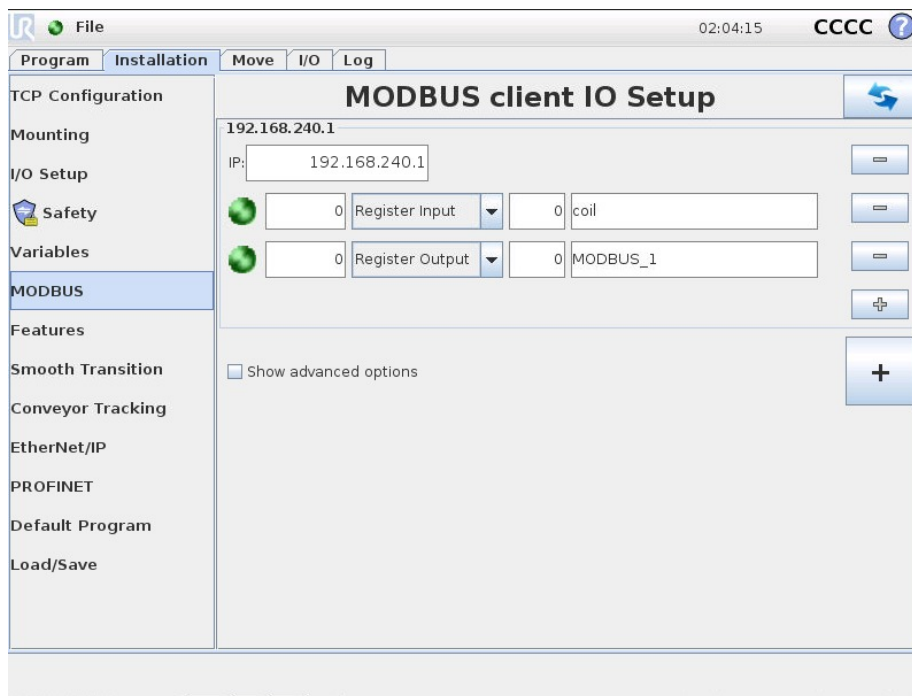


Figura 6: Configurações do Modbus do URSIM

6 Testes e resultados

Os testes integrados validaram com sucesso a arquitetura virtual de intertravamento e a execução da lógica de paletização programada. Através do monitoramento síncrono das plataformas, observou-se a perfeita comunicação em tempo real entre o emissor de comandos e o manipulador cinemático.

Ao inicializar o sistema via URSim, a rotina entra em modo de espera e exige a validação das condições iniciais. O operador interage diretamente com as telas de diálogo nativas do Polyscope, as quais solicitam a liberação dos eixos para a execução do setup inicial de segurança (Freedrive), seguido pela confirmação de inicialização do modo automático. Uma vez habilitada a operação automática e sincronizada com as flags do CLP virtual no CODESYS, o robô inicia os ciclos de movimentação de forma contínua.

Durante a execução das trajetórias, o gêmeo digital estruturado no RoboDK responde instantaneamente aos comandos lógicos, reproduzindo com exatidão a rotina de movimentação da célula industrial. O braço robótico UR10 executa

o deslocamento aéreo até a extremidade da esteira transportadora, realiza a aproximação vertical estritamente perpendicular para a captura da caixa (Pick) e aciona o atuador de garra a vácuo.

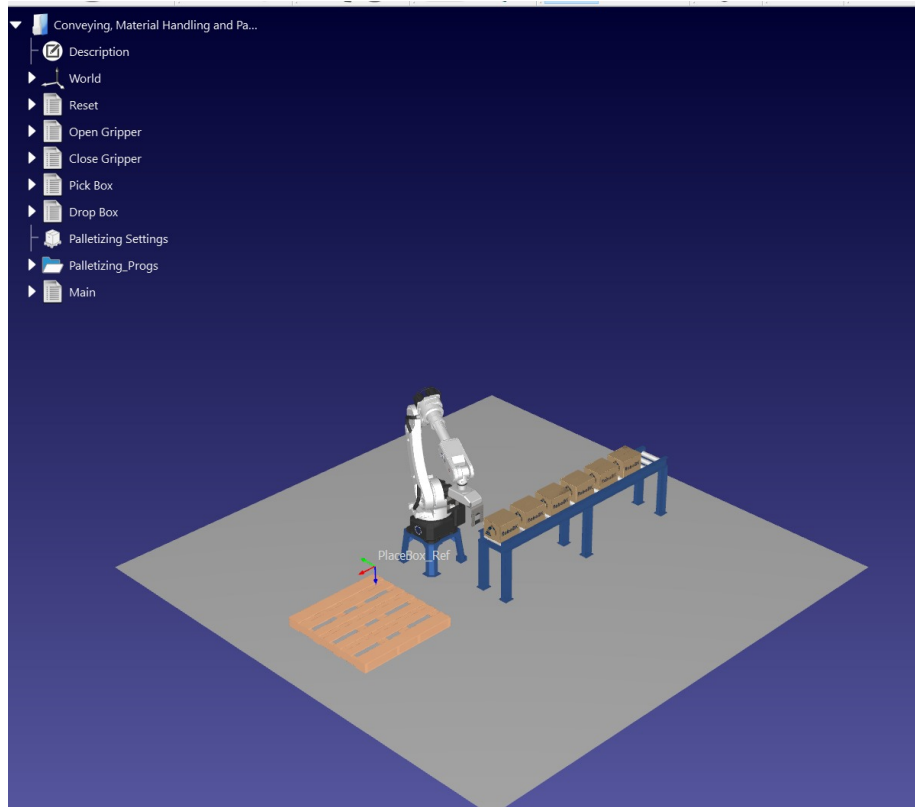


Figura 7: Simulação Robo DK

Em seguida, o robô transporta o material de forma fluida utilizando os raios de concordância configurados e realiza a deposição (Place) na matriz de palletização, respeitando os planos de aproximação calculados dinamicamente. O sincronismo visual entre as juntas físicas emuladas no URSim e o ambiente tridimensional do RoboDK comprovou a eficácia do tráfego de dados via Modbus TCP, operando sem atrasos (delays) ou travamentos na lógica de varredura.

7 Segurança, limitações e melhorias futuras

7.1 Segurança e Intertravamento

Mesmo operando em um ambiente de simulação integrada (CODESYS, URSim e RoboDK), as premissas de segurança ativa foram incorporadas direta-

mente no código-fonte em URScript e na lógica Ladder do CLP. A inicialização da célula exige obrigatoriamente a validação por software do operador através da função `freedrive_mode()` no painel Polyscope, garantindo uma janela temporal segura para o alinhamento físico e setup inicial antes do modo automático.

Durante a operação, o intertravamento lógico monitora constantemente o status das flags via rede: o acionamento de uma parada ou pausa através do registrador `stop_cycle` força a interrupção imediata das trajetórias lineares ou angulares do robô, mantendo, contudo, o nível lógico do atuador de preensão (`gripper_on`) em nível alto para prevenir o desprendimento acidental e a queda de material. Além disso, os planos de aproximação (*Approach Heights*) são calculados dinamicamente em relação ao topo acumulado das camadas do pallet (Z_{layer}), atuando como uma barreira de software contra colisões verticais na matriz de deposição.

7.2 Limitações do Sistema Deduzido

A principal limitação técnica identificada nesta arquitetura virtual está associada à latência de comunicação e ao tempo de varredura (*scan time*) inerentes ao protocolo Modbus TCP quando trafegado sob infraestruturas de rede padrão (Ethernet convencional). Devido à suscetibilidade a flutuações e oscilações na taxa de atualização dos registradores, este protocolo é estritamente restrito ao tráfego de dados e flags lógicas de fluxo de processo (como a sinalização de caixas disponíveis ou ciclos concluídos).

Portanto, o link de rede Modbus TCP não possui determinismo temporal suficiente para trafegar sinais críticos de segurança humana em tempo real. Em uma célula real, funções de emergência (como botões de parada e sensores de intrusão) não dependem da rede e devem ser obrigatoriamente intertravadas por fiação física direta conectada a relés de segurança de canal duplo integrados diretamente ao hardware da controladora do UR10.

7.3 Melhorias Futuras

Como propostas para otimização e evolução deste projeto de paletização, destacam-se os seguintes pontos:

- **Migração de Protocolo de Rede:** Substituir o Modbus TCP por protocolos industriais baseados em Ethernet em tempo real (*Real-Time Ethernet*), como o PROFINET ou EtherCAT. Essa transição reduziria drasticamente o tempo de ciclo de comunicação e traria o determinismo necessário para a rede.
- **Implementação de Segurança Baseada em Rede (PROFIsafe):** Com a adoção de um protocolo determinístico, torna-se viável a integração de chaves de segurança e barreiras de luz diretamente via software utilizando perfis como o PROFIsafe, simplificando o cabeamento físico sem comprometer as normas vigentes (como a ISO 13849-1).

- **Parametrização Dinâmica Avançada via Receitas:** Expandir o uso da palavra `recipe_select` para permitir que o CODESYS envie dinamicamente diferentes layouts de paletização para o URScript, permitindo que a mesma célula processe caixas de dimensões variadas e padrões de empilhamento distintos sem a necessidade de reescrever o código do manipulador.
- **Integração de Sensores Inteligentes na Simulação:** Adicionar a emulação de sensores de medição de vácuo analógicos para monitorar a eficiência da preensão da garra e sensores de barreira adicionais na esteira para o controle de fluxo *upstream* de alimentação das caixas.

8 Conclusões

Em suma nesse trabalho se propôs em desenvolver a simulação e também executar na prática paletização simples no UR10 real com integração à CLP do laboratório. Para tal o URSIM foi o mestre enviando os outputs e esperando inputs do CODESYS. Também se fez a simulação em que passou a trajetória de paletização em tempo real para o ROBO DK partir de um código no URSIM.

Desse modo, usou-se o CLP virtual do CODESYS como server que comunica pelo protocolo ModBus TCP com o cliente que é o URSIM. E foi criada uma simples rotina de paletização de no máximo duas camadas, esse valor é determinado pelo operador. Feitas as devidas configurações para que comunicação acontecesse, então, fez-se a simulação com o ROBO DK.

Mesmo em ambiente de simulação integrada, o projeto incorporou medidas de segurança ativa tanto no código quanto na lógica de controle. Nas quais exige a validação do operador antes da inicialização e o monitorando contínuo do estado do sistema prevenindo acidentes. Garantindo, pelo intertravamento, paradas imediatas em caso de falhas e cálculo das alturas de aproximação para evitar colisões.

A principal limitação identificada na simulação foi na latência e o tempo de varredura do protocolo Modbus TCP. Esse não oferece comunicação determinística em tempo real, tornando o sistema suscetível a atrasos e flutuações. Assim, não é recomendado que as funções de segurança (como botões de parada e sensores de intrusão) não dependem da rede no ambiente real. Para isso, necessita que sejam intertravadas por fiação física direta conectada a relés de segurança de canal duplo integrados diretamente ao hardware da controladora do UR10.

No entanto para superar essa limitação da comunicação seja por rede, é proposto migrar para protocolos industriais como PROFINET ou EtherCAT, garantindo maior precisão e segurança. Também é recomendada implementação da segurança baseada em rede (PROFIsafe), que permite integração de barreiras e chaves via software conforme normas ISO. Outras melhorias incluem a parametrização dinâmica via receitas, para adaptar automaticamente layouts de paletização, e a integração de sensores inteligentes na simulação, ampliando o monitoramento da eficiência e do fluxo de materiais.